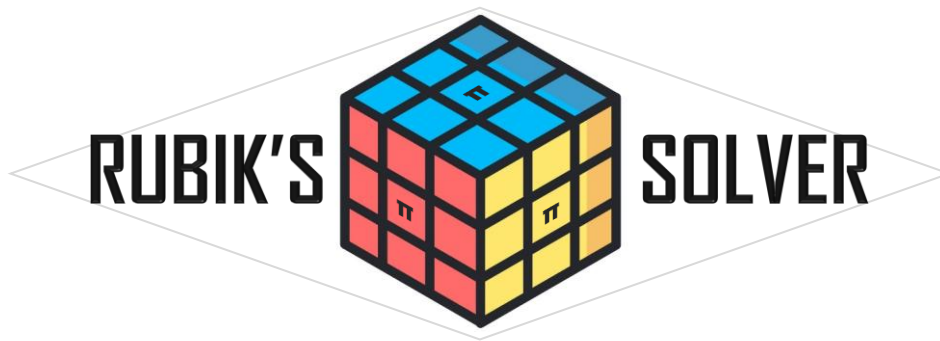


SYSC3010
Computer Systems Development Project

π^3



Group L3-G6

Luke Grundy, 101268449
Basil Thotapilly, 101313150
Eric McFetridge, 101310942
Saim Hashmi, 101241041

TA: Gabriel Seyoum

2/26/2026

Table of Contents

1	Problem Statement	4
1.1	Functional Requirements	4
2	Design Overview	4
2.1	System Overview Diagram	4
2.2	Communication Protocols	5
2.2.1	Communication Protocol Table	6
2.3	Message Sequence Diagram(s)	6
2.3.1	Message Sequence Diagram 1: xxx	7
2.4	Database Table Design/Schema	7
3	Software Design	8
3.1	Software Design for Solver Algorithm Node	8
3.1.1	Node Responsibilities	8
3.1.2	Software Architecture	9
3.1.3	Program Flow	9
3.1.4	Cube representation	9
3.1.5	Move execution	9
3.1.6	Solver Engine	10
3.1.7	Variant Generation	10
3.2	Software Design for Database & GUI Node	10
3.2.1	Node Responsibilities	10
3.2.2	Database Architecture	10
3.2.3	Data Flow and Integration	11
3.2.4	GUI Application Architecture	11
3.2.5	Error Handling and Fault Tolerance	12
3.3	Software Design for Scanner Node	12
3.4	Software Design for Motor Control Node	13
4	Hardware Design	15
4.1	Raspberry Pi and Sense HAT	15
4.2	Motor Control Subsystem	16
4.3	Individual motor module	17
4.4	Pi Camera Module	17
5	GUI Design	18

Commented [JG1]: Update the table of contents before creating the final PDF

5.1	Dashboard Overview	18
5.2	Solve Results Page	18
5.3	Execution Monitoring Page	19
5.4	Solution Review Page	20
5.5	System Logs Page.....	21
6	Test Plans	22
6.1	End-to-end Communication Demo Test Plan	22
6.2	Unit Test Demo Test Plan	22
6.2.1	Scanner Pi Unit Tests.....	22
6.2.2	Solver Algorithm Pi Unit Tests.....	23
6.2.3	Motor Control Pi Unit Tests	23
6.2.4	Database Unit Tests	23
6.2.5	GUI Unit Tests.....	23
6.3	Final Demo Test Plan	23
7	Project Update	23
7.1	Project Milestones.....	24
7.2	Schedule of Activities	24

1 Problem Statement

The objective of this project is to develop a system that can solve a Rubik's cube by scanning the cube, finding a solution, then executing that solution. By using digital cameras, the first module will scan each face and convert the 3x3x3 layout to a data structure stored in the systems local database. Using this data structure, the second module will produce a valid solution for the current cube state. This solution is converted to a series of ninety-degree rotations and will also be stored in the local database. Another module, the motor control system, takes these rotations and executes them in order. The motor layout of the device allows the system to spin any 5/6 faces on the cube. This allows the device enough control over the cube to solve it as well as the ability to easily slide the cube in and out of the housing. On top of solving the cube, this device will help to teach the user the concepts that explain how it functions. By allowing the user to adjust parameters like which solving algorithm to use, how fast to spin the cube, and providing a simple software interface, the system no longer has a single purpose but can provide an interactive learning platform to exercise the users' brain.

1.1 Functional Requirements

Cube Scanning: The system shall recognize the state of the input Rubik's cube using 2 cameras and send it to the server as a data structure that can be stored in the database.

Solving algorithm: The system shall retrieve the cube state from the DB and generate a solution using the selected algorithm.

Solving Output: The system shall use the CFOP method for solving and send the solution to the server in the form: Move # | Face # | Direction.

Cube permutations: The system shall use 5 motors to rotate the cube based on commands received from server module. The system should solve the cube in less than 15 seconds.

System GUI: The system shall display a 3D cube model and solving steps on a web dashboard accessed by a phone or computer.

Notification system: The system shall send a notification to a connected device in the event of a fatal error (disconnected motor/camera, unresponsive pi)

Solving Pipeline: The system shall process the data in the following order: Scan, Solve, Rotate, Verify.

Connectivity: The system shall be able to send data between raspberry Pis using TCP.

2 Design Overview

2.1 System Overview Diagram

Figure 1 presents the high-level deployment of Rubik's Cube Solver. The system is intentionally distributed across multiple Raspberry Pis to keep each subsystem modular and independently testable. The Cube Scanning Pi is responsible for image capture and cube-state extraction; the Solver Algorithm Pi generates a valid solution sequence, and the Motor Control Pi performs time-critical actuator control. A dedicated Database & GUI Pi hosts a centralized database for shared state, error logging, and coordination, as well as the web-based interface used by the user.

This structure reduces coupling between subsystems and improves debugging. Each Pi publishes its outputs to the database (e.g., cube state; solution moves, motor progress) and consumes only the data needed for its next step. The Database & GUI Pi acts as the “single source of truth” for the pipeline state (Scan → Solve → Execute), allowing the system to recover more cleanly from partial failures and making it easier to verify integration milestones.

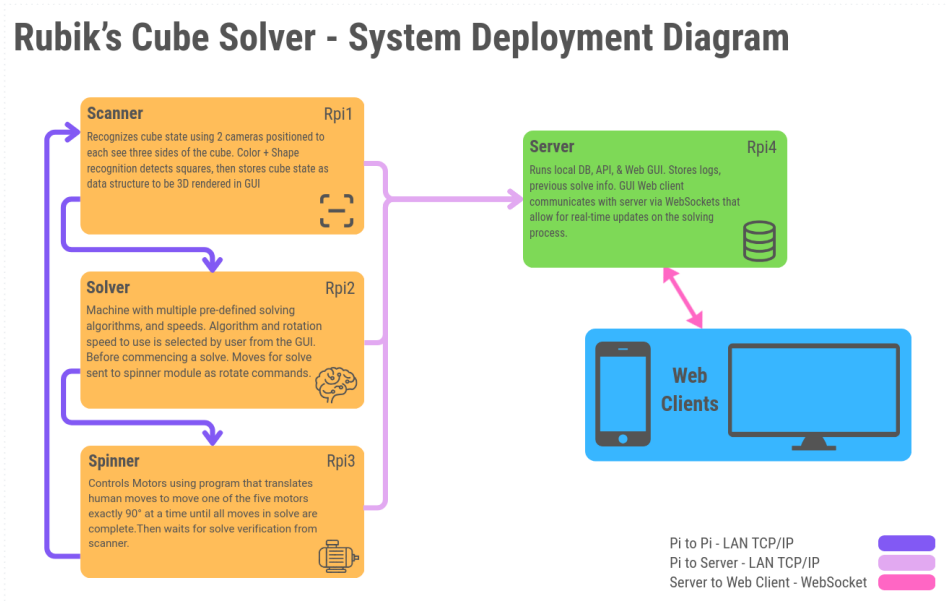


Figure 1: Deployment diagram

Commented [JG2]: Each figure must have a figure. Use automatic numbering. For MS Word, use "References-->Insert Caption"

2.2 Communication Protocols

All Raspberry Pis communicate over the local Wi-Fi network using TCP/IP for guaranteed local data delivery with minimal overhead. Each pi simply has a dedicated TCP socket where info is sent and received by the server. The system does not need to be exposed to the internet to function, unless the user desires to port forward their web dashboard for outside access. System coordination is implemented through a centralized database hosted on the Database & GUI Pi. Each subsystem reads and writes structured records to the database to exchange information and to signal progress.

Inter-node communication (Pi ↔ Pi via Database & GUI Pi)

- Cube Scanning Pi → Database: Writes the detected cube state for the active job, including a timestamp and a confidence/validation flag.
- Solver Algorithm Pi ↔ Database: Reads the latest valid cube state, writes back the computed solution (move sequence) plus metrics such as compute time and move count.

- Motor Control Pi ↔ Database: Reads the solution moves (or a command queue) and writes back live execution status (current move index, completion status, and any error codes).
- All Pis → Database: Write periodic heartbeat timestamps so the system can detect failures quickly.
- Database & GUI Pi → User GUI: The web server reads the current job state from the database and displays live status. GUI actions (Start/Stop/Reset/Rescan) are written as control signals for other subsystems to observe.

Hardware communication (Pi ↔ Devices)

- Cameras ↔ Cube Scanning Pi: Two cameras connect via USB (or CSI) and are accessed through the camera driver stack. Frames are processed locally to detect sticker colors.
- Motors ↔ Motor Control Pi: Motors are driven through motor driver modules controlled by the Pi's GPIO. The motor controller uses STEP/DIR (or equivalent) signals and enforces safe motion limits and stops behavior.

Message format and ownership to avoid conflicts, each subsystem “owns” the data it produces:

- Scanning Pi owns cube state records.
- Solver Pi owns solution records.
- Motor Pi owns execution status records.
- Database & GUI Pi owns the job state machine, logging, and notification triggers.

2.2.1 Communication Protocol Table

Four primary communication edges can be identified in the Rubik's Cube Solver system. All Raspberry Pi nodes communicate over a local Wi-Fi network using TCP/IP and coordinate through a centralized database hosted on the Database & GUI Pi. Each node exchanges structured records through shared database tables.

The Cube Scanning Pi writes detected cube states to the database once image capture and colour classification are complete. The Solver Pi reads validated cube states, computes a solution sequence, and writes the resulting move list back to the database. The Motor Control Pi reads the solution moves and updates execution progress in real time as rotations are performed. The GUI component reads job state information from the database and writes control commands such as Start, Reset, or Rescan, which are observed by the other subsystems.

2.3 Message Sequence Diagram(s)

Each Message Sequence diagram shows a series of messages (listed in the Communication Protocol Table) sent between nodes (shown in the Deployment diagram). A single diagram depicts one scenario (use case) handled by the system. You will likely require multiple scenarios to cover all major behaviours of your system, so you will require multiple Message Sequence Diagrams. These sequence diagrams illustrate examples of how the messages listed in your Communication Protocol Table are used to accomplish goals, where the nodes work together as a system.

2.3.1 Message Sequence Diagram 1: xxx

Describe each scenario/use case and provide a message sequence diagram for each. As above, each figure must be referenced and described in the text using a cross-reference (hyperlink). Figure 2 illustrates the first use case, where...

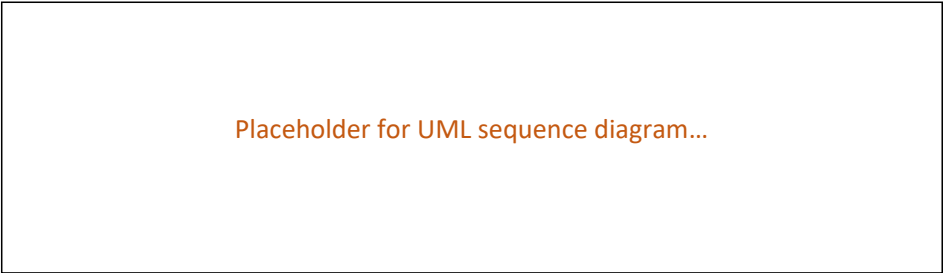


Figure 2: Sequence diagram for use case 1: xxx

2.4 Database Table Design/Schema

The Database & GUI node hosts a relational database that serves as the central “source of truth” for the Rubik’s Solver system. Instead of having nodes send direct messages to each other, each subsystem reads from and writes to shared tables. This keeps the system loosely coupled: the scanner can upload results even if the solver is temporarily unavailable, and the motor controller can continue execution using stored steps without relying on live commands. The database also makes debugging easier because intermediate results (scan output, solution steps, and execution logs) are preserved and can be reviewed after a run.

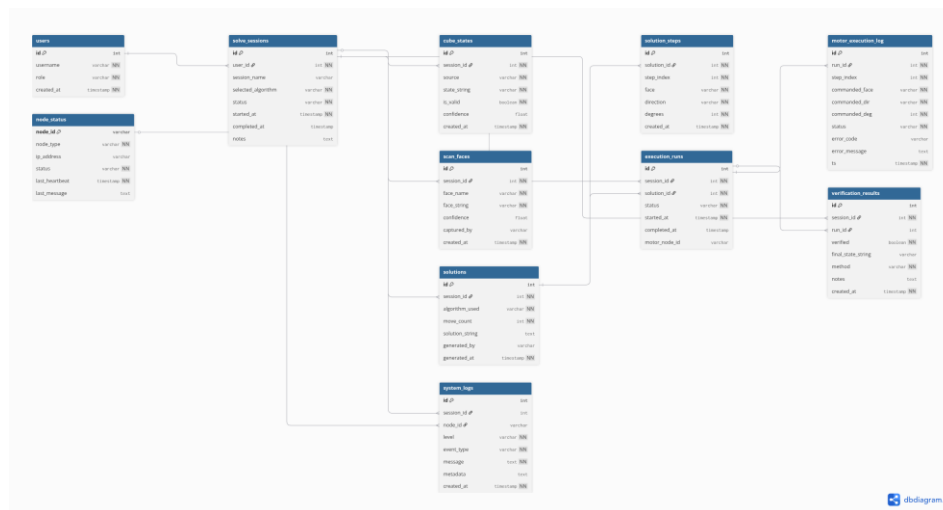
shows the full schema used to coordinate the pipeline **Scan → Solve → Execute → Verify**, while also supporting system monitoring and logging. At a high level, each “solve attempt” is represented by a row in [solve_sessions](#). A session stores the user who initiated it, the selected algorithm, timestamps, and an overall status. The scanned cube information is stored in two layers: [scan_faces](#) stores per-face outputs (U, R, F, D, L, B) and confidence values, while [cube_states](#) stores the final 54-character cube representation once the scan is complete. Separating face-level results from the final cube state makes the scan stage easier to validate and debug, since the GUI can display which face was weak or inconsistent before committing to solving.

Once a valid cube state exists for a session, the solver produces a solution stored in solutions. The solution is broken into ordered instructions in [solution_steps](#), where each step has a [step_index](#) to enforce a deterministic move sequence. This table format also makes it easy for the GUI to display the full move list and highlight the current step during execution. Execution is intentionally separated from solution generation using [execution_runs](#). A run represents an attempt to physically execute a solution (including retries), and it can be paused, aborted, or failed without deleting the original solution. During execution, the motor subsystem logs progress into [motor_execution_log](#), recording acknowledgements, completion states, and error information per step. This gives clear traceability when a solve fails mid-way (for example, if a motor stalls or a command is not acknowledged).

After execution, the system records the outcome in [verification_results](#), including whether the cube was confirmed solved and (optionally) the final cube state if it was re-scanned or manually verified. This table

supports both the “Verify” stage of the pipeline and post-run reporting in the GUI. For reliability monitoring, node health is tracked in `node_status` using a heartbeat timestamp and status field, allowing the GUI to show whether each Raspberry Pi is online and responsive. Finally, `system_logs` stores timestamped events (info/warning/error) linked to both session and node where applicable, which helps with troubleshooting and supports later analysis of recurring faults.

Overall, this schema is designed to (1) preserve system state across failures, (2) support clear ordering and traceability of moves, (3) make runs reproducible through stored sessions and solutions, and (4) provide enough logging and monitoring data to debug integration issues during development and demos.



Database Entity-Relationship Diagram

3 Software Design

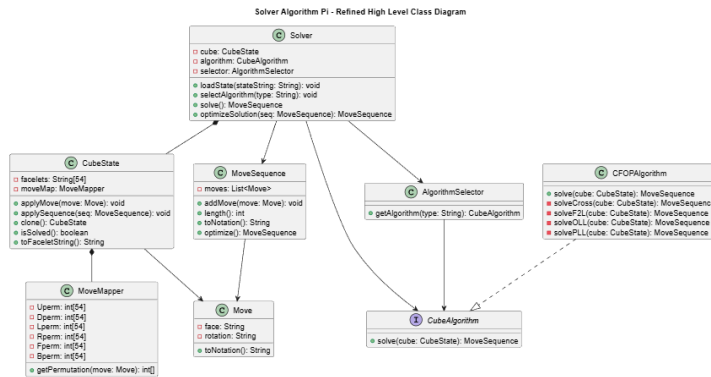
3.1 Software Design for Solver Algorithm Node

3.1.1 Node Responsibilities

The Solver Algorithm Pi is responsible for generating a valid move sequence from a scanned Rubik’s cube. It will receive a solve request from the GUI/Database Pi and generates a string of commands to be sent back to the GUI/Database Pi to be used elsewhere. As with all components, when the solver is not in use it will send a heartbeat to confirm the Pi is connected and ready. When the solver receives a request, it will receive two inputs: the current cube state and the selected algorithm.

3.1.2 Software Architecture

The Software Architecture uses multiple classes to maximize modularity and testability. These classes fit into three different categories: Top layer interaction, Cube state/Permutation, State identifying/Algorithm solver.



3.1.3 Program Flow

The Solver Algorithm Pi operates in a continuous loop:

1. Check for job request
2. Retrieve Cube state
3. Compute solution
4. Validate cube
5. Optimize move sequence
6. Store solution in database
7. Update heartbeat

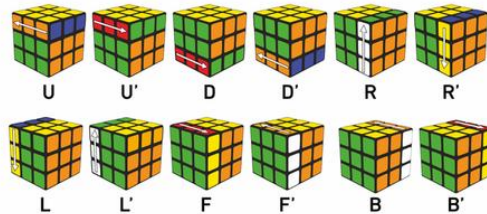
Cube is validated during solution so that if the algorithm reaches a point where it is unable to solve the cube it can return an invalid state. This approach removes the need for a cube validator.

3.1.4 Cube representation

The cubes state is represented using a 54-facelet array. This means that every 9 values stored in the array represents one face. Each face is stored by storing the face stickers from left to right, top to bottom, with the 5th value in each face being the center of the face.

3.1.5 Move execution

To execute moves on the cube representation, a movement map is made. This map gives the indices of where each sticker will end up based on their position and the instruction.



3.1.6 Solver Engine

The solver implements the CFOP method to solve the cube (Cross, F2L (first Two Layers), OLL (Orient Last Layer), PLL (Permute Last Layer)). This stage will operate on a cloned cube state and find the needed algorithm based on its state. The Algorithm will then be saved to a move set string and performed on the cloned cube. This will result in the algorithm being saved to perform on the real Rubik's cube and the state of the cube after the algorithm on the cloned cube. The process is then repeated for the next solve phase until the cube is completely solved.

3.1.7 Variant Generation

To reduce total move count, the solver generates multiple solution variants. This is done by performing alternative algorithms to the same cube and comparing the move counts between each variant.

3.2 Software Design for Database & GUI Node

3.2.1 Node Responsibilities

The Database & GUI node functions as the central coordination layer of the Rubik's Solver system. While the Scanner, Solver, and Motor Control nodes perform specialized tasks, this node is responsible for maintaining system state, storing intermediate and results, and presenting information to the user through a web-based dashboard.

The database acts as the single source of truth for the entire solving pipeline. All nodes communicate indirectly through it. This reduces tight coupling between subsystems and simplifies debugging during integration. The GUI provides visibility into the state of the system, including cube detection results, generated solutions, execution progress, and node health.

3.2.2 Database Architecture

The database functions as the central coordination mechanism for the entire Rubik's Solver system. Rather than allowing direct communication between nodes, all subsystems interact through shared database tables. This design reduces tight coupling between components and ensures that each node can operate independently while maintaining a consistent system state.

At a high level, each solve attempt is represented as a session. A session progresses through defined stages (scanning, solving, executing, verifying), with intermediate and results stored persistently. The database stores the detected cube state, the generated solution, the ordered execution steps, and the final verification outcome. This structured progression ensures that each stage of the pipeline can be monitored, resumed, or debugged without restarting the entire system.

Execution is treated as a separate process from solution generation. This allows the system to retry or analyze a run without modifying the original solution data. In addition to solve-related data, the database also maintains node heartbeat information and system logs. This supports real-time monitoring through the GUI and improves fault detection during integration and testing.

The detailed relational schema, including table definitions and relationships, is provided in [Section 2.4](#).

3.2.3 Data Flow and Integration

The database supports the following data flow:

1. The Scanner node writes individual face detections and the final 54-character cube state.
2. The Solver node retrieves the cube state, computes the solution, and stores ordered solution steps.
3. The Motor Control node retrieves the solution steps and writes execution logs during rotation.
4. After execution, verification results are stored and displayed in the GUI.
5. All nodes periodically update their heartbeat status.

This approach ensures that if one subsystem fails, the system state remains preserved in the database. Recovery and debugging are therefore simplified.

3.2.4 GUI Application Architecture

The GUI is implemented as a web-based dashboard hosted on the same Raspberry Pi as the database. The application follows a client-server structure:

- The backend handles database queries and API endpoints.
- The frontend renders session data, node status, and execution progress.

The GUI is organized into the following main views:

- Dashboard: System overview and recent sessions
- New Solve: Cube scanning and configuration
- Solution Review: Generated move list
- Execution Monitoring: Real-time motor progress
- History: Past solve sessions
- System Status: Node health and heartbeat data
- Logs: Event-level debugging information

The interface prioritizes clarity and operational visibility rather than visual complexity. Status indicators and progress bars are emphasized to support real-time monitoring during demonstration.

Detailed wireframes for each screen are provided in [Section 5](#).

3.2.5 Error Handling and Fault Tolerance

The Database & GUI node includes several mechanisms to handle faults:

- Node heartbeat tracking to detect disconnected Raspberry Pis
- Execution log tracking to detect incomplete rotations
- Verification step to confirm successful cube resolution
- Structured logging for scan, solve, and execution events

If a node fails to update its heartbeat within a defined timeout period, its status is marked as offline in the database and displayed in the GUI.

All solve sessions maintain a status field that transitions through defined states (created, scanning, solving, executing, verifying, complete, failed). This prevents ambiguous system states during debugging.

3.3 Software Design for Scanner Node

The Scanner Node is responsible for capturing images of assigned cube faces, extracting sticker colours, formatting the detected state, and publishing the results to the central database. The scanner is implemented with 3 raspberry Pis equipped with CSI camera modules that each execute the same python script to scan the cube data. Scanning occurs only during the initialization phase of the solving process and is inactive afterwards until triggered again.

Upon execution the Scanner will perform the following operations:

1. Initialize the CSI camera module using the Picamera2 interface.
2. Capture a high-resolution image of the cube face.
3. Apply preprocessing operations (cropping, normalization, and noise reduction).
4. Divide the image into a 3×3 grid corresponding to cube stickers.
5. Compute the average RGB value of each sticker region.
6. Classify each sticker into one of six cube colours using calibrated thresholds.
7. Format the detected face into a structured 54 character string.
8. Publish the face data to the shared database.
9. Confirm successful write before terminating.

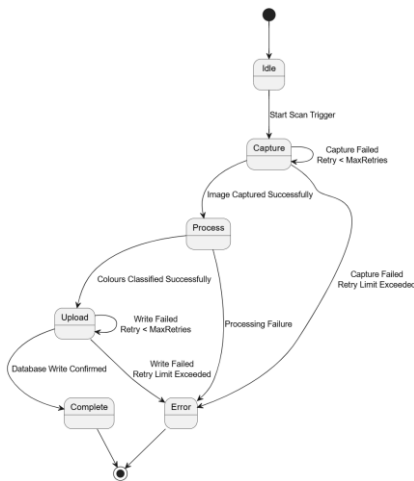
The Scanner Node follows a simple state-based execution model:

Idle -> Capture -> Process -> Upload -> Complete

- Idle: wait for start command.
- Capture: Acquire image from camera.

- Process: Perform grid extraction and colour classification.
- Upload: Send formatted face data to database.
- Complete: Exit or return to idle.

If an error occurs during capture or upload, the system transitions back to the same state with a retry limit.



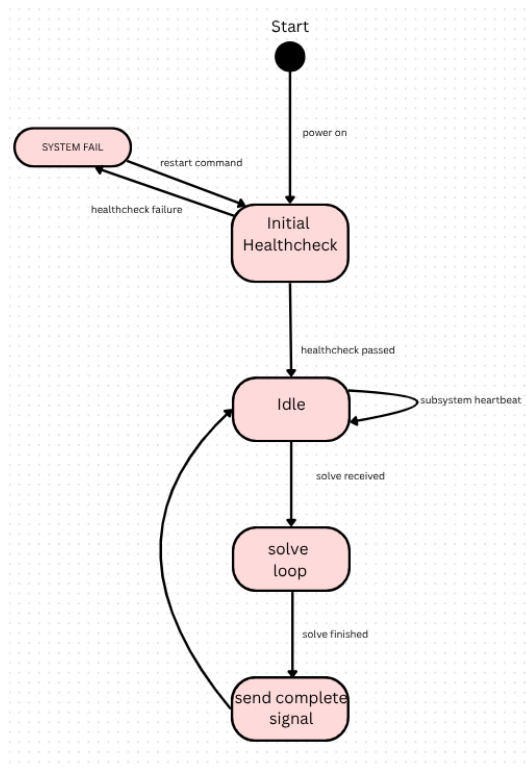
The image processing pipeline is implemented using OpenCV and NumPy. The captured image is first resized and cropped to isolate the cube face. A Gaussian blur is applied to reduce lighting noise. The processed image is then divided into nine equal square regions. For each region A centered subregion is sampled to avoid edge reflections, then the mean RGB value is calculated and the RGB vector is compared against calibrated reference ranges and assigned to the cube colour it is closest to. The reference ranges will be determined under fixed lighting conditions and stored in the script as threshold parameters.

The Scanner communicates directly with the central database hosted on the GUI/database Pi. When a scan is complete an entry is written into the cube state table containing job_id (unique solve session identifier), state_string (54 character string representing the cube state), confidence (a float indicating reliability), is_valid (a boolean confirming validity) and ts (a timestamp).

3.4 Software Design for Motor Control Node

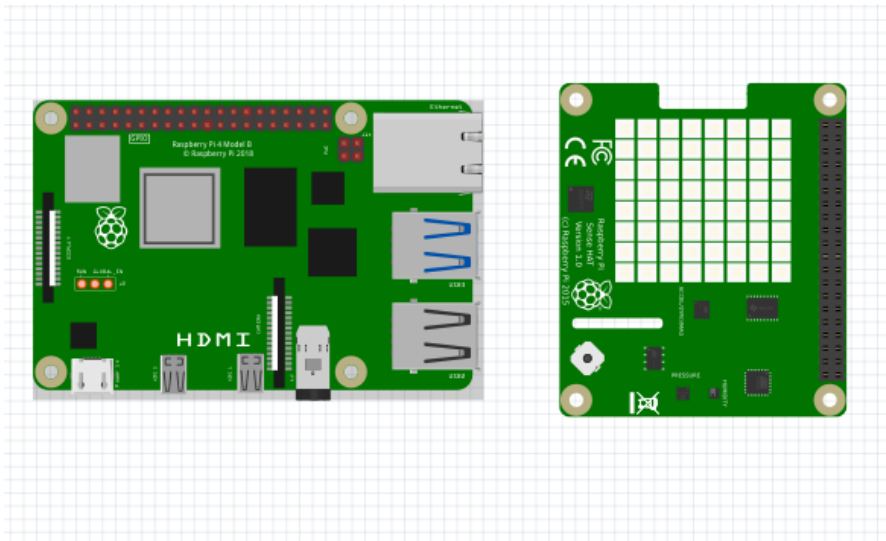
The motor control software sits dormant for the initial part of the solving process, just sending a health check heartbeat to confirm the system is present and ready. Upon completion of the solve by the solving Pi the solve will be sent to the server Pi to be stored in the database. Once the solving pattern for that

scramble has been stored, the entire move pattern is copied to a local buffer on the RPi. This ensures that we are never waiting on commands from the server to proceed, we already have the entire list of moves we need to solve the cube. We utilize the pigpio library for the stepper motor drivers. Which allows us to drive the motors at 10+ kHz speeds. We are aiming for between 0.1, for 0.2 seconds per move which means we require an RPM of approximately 120 for each motor at peak. The pigpio library runs a daemon on the pi that gives us real-time hardware access, so the OS does not interfere with the motor timing. This pigpio library uses C for the real-time parts, but we can write all our control scripts in python while still benefiting from the highspeed gpio. This means that the motor control subsystem can be written completely in python. We use smooth acceleration curves for the motors to ensure they move accurately. The motor loop runs for the duration of however many moves the current solve requires. Then the system sends a solve complete signal back to the server, and proceeds to idle again at low power until the next solve.



4 Hardware Design

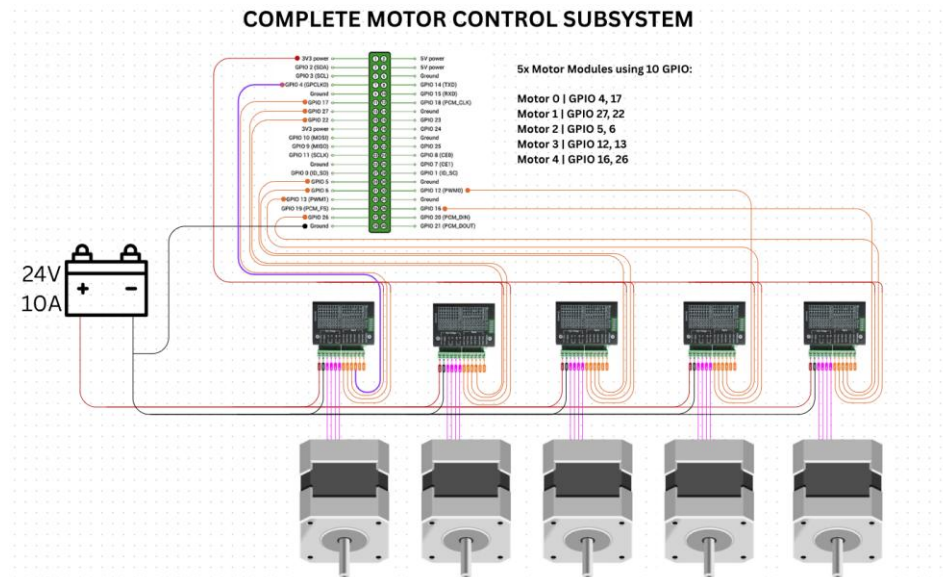
4.1 Raspberry Pi and Sense HAT



Rpi and Sense HAT - schematic view

The Raspberry Pi and Sense HAT modules are at the heart of the cube solving system. Each individual subsystem composes of a Raspberry Pi and its attached Sense HAT. The Pi's provide the gpio and processing that makes each of the individual parts possible. They also contain the networking stacks needed for inter-Pi communication. The Pi's are extremely versatile, being used for optical recognition, cube solving, hardware control, and as a server all just within this one project. The Sense HAT's provide real-time monitoring of every pi and includes a gpio passthrough that allows us to still take advantage of all the free pins on the board.

4.2 Motor Control Subsystem

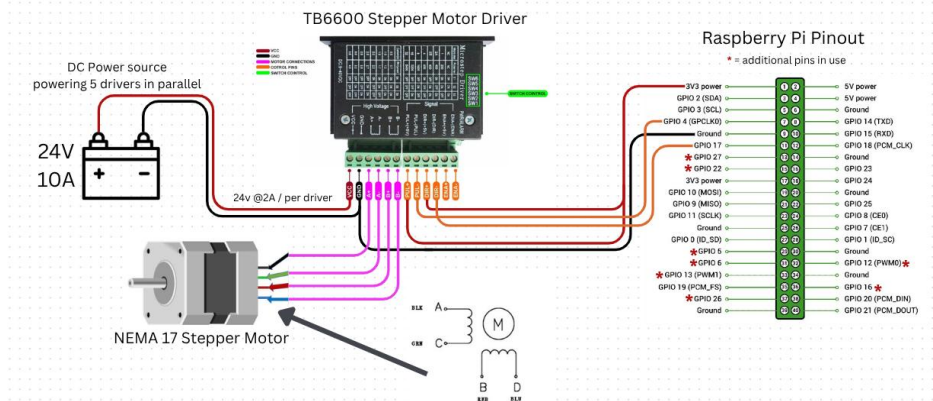


Motor control subsystem - schematic view

The hardware design for the motor control system features 5 Nema 17 motors operating at a maximum level of 24v and 2A. This will ensure the motors have more than enough torque to rapidly snap the cube into place, as well as hold tight while the other motors turn. Using 10 Gpio pins on the Pi we can individually step each of the 5 motors using the pigpio library in pair with the tb6600 model stepper drivers. These drivers are rated for up to 4A and can easily deliver the power needed for our cube solver. The driver steps the motor when the control line is pulled low and changes the direction when the direction line is pulled low. All drivers are connected to a common power source and the 3.3v power rail on the raspberry pi. The Pi ground is also connected to the ground of the external power source to have a single ground for the whole system.

4.3 Individual motor module

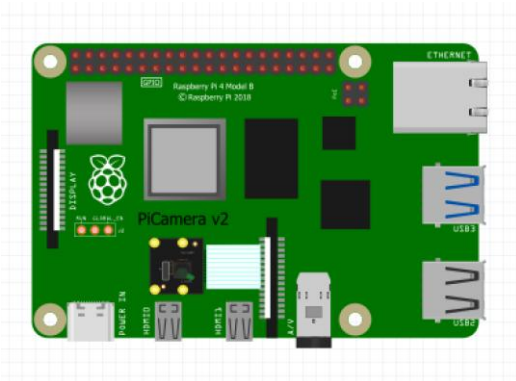
Pi Motor Control Subsystem: Individual Motor Unit (5 in full system)



Individual motor unit - schematic view

This is a zoomed in view of one of the five motor modules in the cube solver. Here you can see how each individual motor is wired back to the Pi. The other pins used by the motors are marked with red stars. These pins were chosen as not to collide with the Sense HAT. Allowing it to still be used for environmental monitoring.

4.4 Pi Camera Module



Pi Cam – schematic view

The Pi Camera Module is part of the Scanner Subsystem and is responsible for capturing images of the Rubik's Cube faces for colour detection. Each camera is connected directly to a Raspberry Pi with a CSI

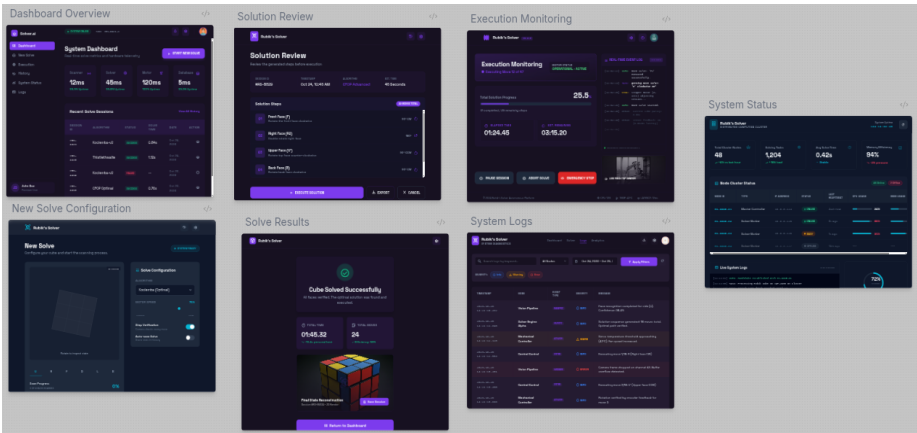
ribbon cable. The camera is controlled using the Picamera2 library in python. The camera is powered directly through the CSI connection and does not require any external circuitry. It operates at 3.3V supplied by the Raspberry Pi. There will be three cameras to accurately view all faces of the cube which means three of the Pis will be used to determine the cube state.

5 GUI Design

5.1 Dashboard Overview

The Dashboard Overview page serves as the landing screen for the system. It provides a high-level summary of system health and recent activity. The status cards at the top display the operational state of each node (Scanner, Solver, Motor Control, and Database) using real-time heartbeat data stored in the `node_status` table. This allows the user to immediately determine whether all subsystems are online before initiating a solve.

The lower portion of the dashboard displays recent solve sessions retrieved from the `solve_sessions` table. Each entry includes session status, selected algorithm, and completion time. From this page, the user can initiate a new solve or review previous sessions, making it the central navigation hub of the application.

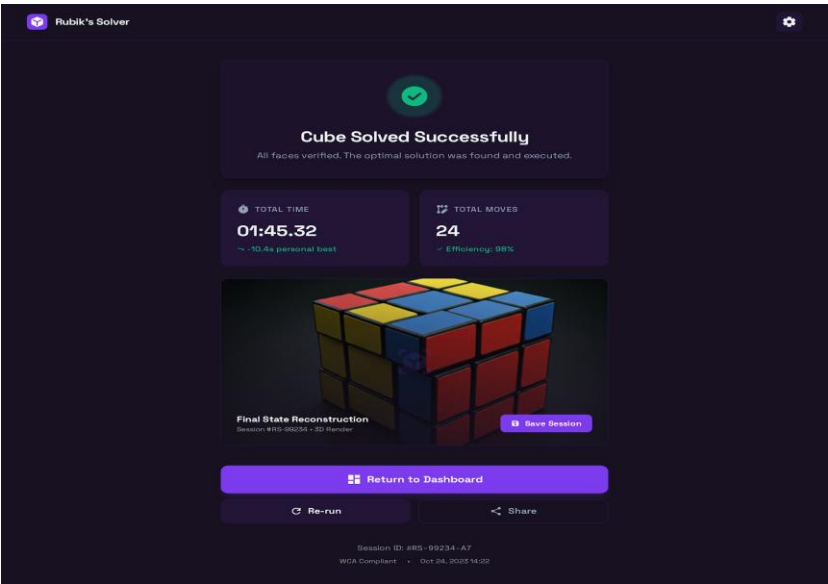


Initial Wireframes designed for GUI of the application

5.2 Solve Results Page

The Solve Results page presents the final outcome of a solve session. Once execution completes, the verification stage writes the result to the `verification_results` table. This page retrieves the final status (success or failure), total time, and move count from the database and displays a reconstructed cube state for confirmation.

The user may choose to save the session, return to the dashboard, or re-run the solution if needed. By separating results from execution monitoring, the interface provides a clear end-state summary suitable for demonstration and evaluation.

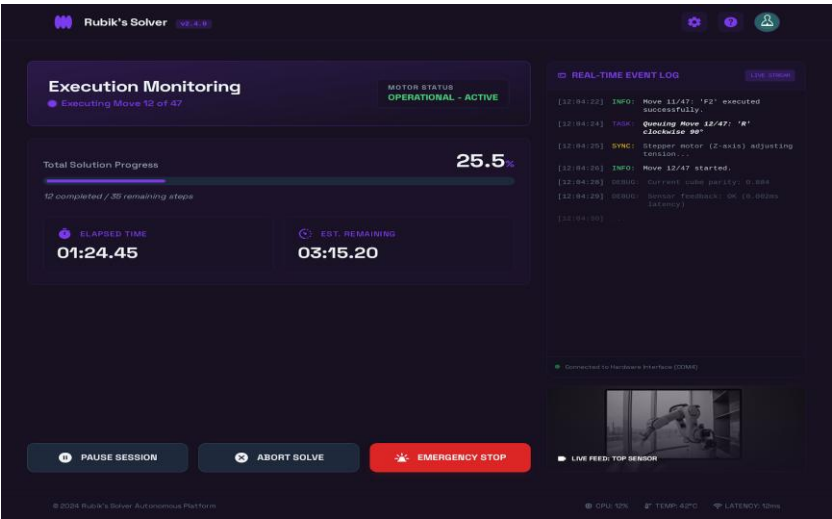


Solve results page

5.3 Execution Monitoring Page

The Execution Monitoring page provides real-time visibility into the physical solving process. As the Motor Control node performs each rotation, it logs progress to the motor_execution_log table. The GUI continuously queries this table to update the progress bar, current move indicator, and event log.

The page also displays elapsed time, estimated remaining time, and system status. Control buttons such as Pause, Abort, and Emergency Stop allow the operator to intervene if necessary. This interface is designed for demonstration and debugging, offering transparency into low-level motor behavior during execution.

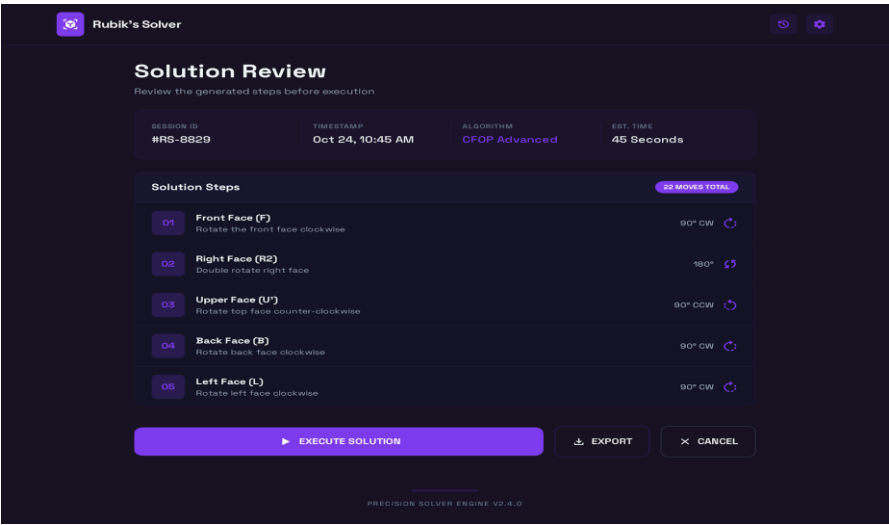


Execution Monitoring page

5.4 Solution Review Page

The Solution Review page displays the generated move sequence before physical execution begins. Information shown on this page is retrieved from the solutions and solution_steps tables. The move list is ordered using the `step_index` field to preserve deterministic execution order.

This screen allows the user to verify the selected algorithm, estimated solve time, and total move count before committing to motor execution. By separating review from execution, the system reduces the risk of unintended motor movement and provides a clear checkpoint in the solving pipeline.

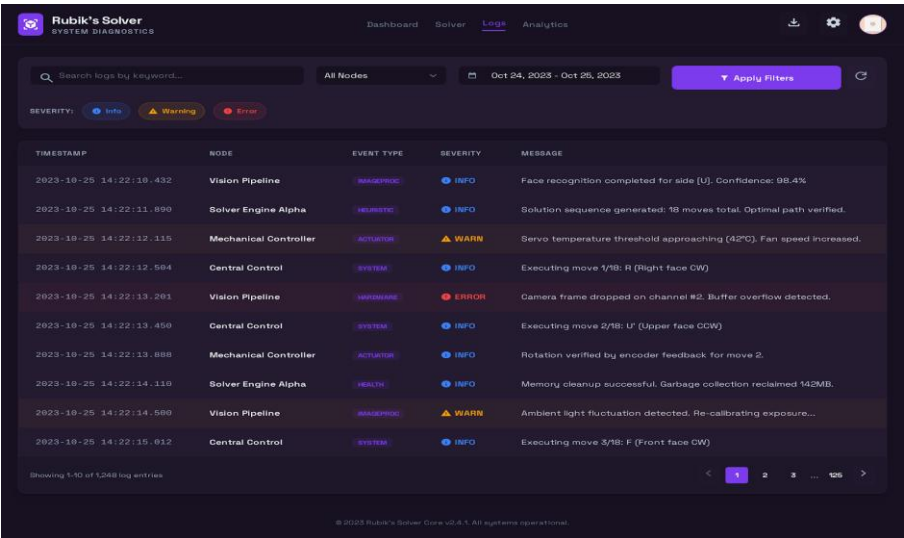


Solution Review page

5.5 System Logs Page

The System Logs page provides detailed visibility into system-level events. Log entries are retrieved from the system_logs table and include timestamp, node identifier, event type, severity, and message content. Filtering options allow the user to isolate specific nodes or error levels.

This page is primarily intended for debugging and integration testing. During development, it allows the team to trace communication issues, scanning inconsistencies, or motor faults without directly accessing the database. It also supports demonstration of the system’s fault-detection capabilities.



The screenshot displays the 'Rubik's Solver SYSTEM DIAGNOSTICS' interface. At the top, there are navigation tabs for 'Dashboard', 'Solver', 'Logs' (which is active), and 'Analytics'. Below the tabs is a search bar labeled 'Search logs by keyword...' and a filter dropdown set to 'All Nodes'. A date range selector shows 'Oct 24, 2023 - Oct 25, 2023'. A 'SEVERITY' filter bar includes 'Info', 'Warning', and 'Error' options. The main area is a table of log entries with columns: 'TIMESTAMP', 'NODE', 'EVENT TYPE', 'SEVERITY', and 'MESSAGE'. The table contains 10 entries, with the first 9 visible. The last entry is partially obscured by a pagination bar showing 'Showing 1-10 of 1,048 log entries' and a page number '1'.

TIMESTAMP	NODE	EVENT TYPE	SEVERITY	MESSAGE
2023-10-25 14:22:10.432	Vision Pipeline	MESSAGE	INFO	Face recognition completed for side (U). Confidence: 98.4%
2023-10-25 14:22:11.890	Solver Engine Alpha	MESSAGE	INFO	Solution sequence generated: 10 moves total. Optimal path verified.
2023-10-25 14:22:12.115	Mechanical Controller	ACTION	WARN	Servo temperature threshold approaching (42°C). Fan speed increased.
2023-10-25 14:22:12.584	Central Control	SYSTEM	INFO	Executing move 1/10: R (Right face CW)
2023-10-25 14:22:13.201	Vision Pipeline	MESSAGE	ERROR	Camera frame dropped on channel #2. Buffer overflow detected.
2023-10-25 14:22:13.450	Central Control	SYSTEM	INFO	Executing move 2/10: U' (Upper face CCW)
2023-10-25 14:22:13.888	Mechanical Controller	ACTION	INFO	Rotation verified by encoder feedback for move 2.
2023-10-25 14:22:14.110	Solver Engine Alpha	MESSAGE	INFO	Memory cleanup successful. Garbage collection reclaimed 142MB.
2023-10-25 14:22:14.580	Vision Pipeline	MESSAGE	WARN	Ambient light fluctuation detected. Re-calibrating exposure...
2023-10-25 14:22:15.012	Central Control	SYSTEM	INFO	Executing move 3/10: F (Front face CW)

System Logs page

6 Test Plans

6.1 End-to-end Communication Demo Test Plan

The end-to-end test will aim to demonstrate the functionality of the heartbeat system. The heartbeat system ensures the connection of the Pis and will put the system into a safe mode if connection to one of the modules fail.

This test will demonstrate that each raspberry Pi can communicate via TCP by sending and receiving heartbeats. The test requires that all 4 Pis are connected to the local WI-FI with one Pi acting as the database/GUI Pi.

The test will proceed as such:

- All Pis will be turned on and connect to the Pi acting as the Database.
- One Pi will manually be disconnected from the internet
- Wait 5+ seconds
- Database Pi will show the Pi is disconnected.

6.2 Unit Test Demo Test Plan

6.2.1 Scanner Pi Unit Tests

Camera Connectivity test - Standalone script to capture frame from each camera. Error if both cameras don't return an image

Color Detection test - present cube with known pattern, run algorithm, compare results.

6.2.2 Solver Algorithm Pi Unit Tests

Solved State test - Input a solved cube. Output solution should be empty

Valid Scramble test - Input cube with known solution. Output string should solve cube

Invalid Scramble test - Input cube that is unable to be solved. Invalid flag should be thrown

6.2.3 Motor Control Pi Unit Tests

Rotation Accuracy test - Command 1 rotation. Measure result angle

Speed test - Run at full speed for a known number of rotations. Ensure rotation matches.

6.2.4 Database Unit Tests

Insert/Retrieve test - Insert and retrieve data from database. Ensure they match

6.2.5 GUI Unit Tests

State display test - Ensure cube state and the cube display are identical.

Control Functionality – Ensure button flags update when pressed.

6.3 Final Demo Test Plan

For the final demo, plan the scenarios that you will run to demonstrate that each of the functional requirements have been met. Are there any frameworks or preparations needed for these final tests?

Example 1: For a system that is building a smart helmet to prevent concussions, what mechanics are needed to systematically impose a force that is less than, equal to, and greater than the concussion limit?

Example 2: For a bus onboarding system that guarantees a maximum number of occupants, what sequence of passenger arrival is needed to demonstrate an empty bus, a partially loaded bus, a full bus, and a full bus that just became partially loaded?

7 Project Update

Development of the Rubik's cube solver has had a slow beginning. The team has created frameworks for each individual component and has begun developing their respective software. However, the team has missed the first deadline, preventing a comprehensive Communication test. To adapt to this issue, a revised version of the end-to-end unit testing has been created to show the general functionality of the final product.

For the team to get back on track, the deadlines will be shifted to account for the lost time, stricter due dates and regular check ins will occur to ensure that the development of needed components can be completed.

7.1 Project Milestones

Milestone Number	Milestone Name	Description	Deadline
1	Database running	The database Pi is running a local database with the required tables	February 15 th (revised to March 9 th)
2	Inter Pi Communication Established	All Raspberry Pis can successfully read from and write to the shared database.	February 28th
3	Cube Scanning Functional	The scanning Pi can capture images of the cube, detect sticker colours and generate a valid cube state stored in the database.	March 3 rd (revised to March 9 th)
4	Solver Algorithm Integrated	Given a valid cube state from the database the solver Pi can generate a complete sequence of moves that solve the cube.	March 9th
5	Motor Control Execution	The motor control Pi can correctly execute individual cube rotations based on received commands and send confirmation of completion to the database.	March 12th
6	System Verification and Debugging	The fully integrated system can be tested to identify and correct faults across all subsystems	March 17th

7.2 Schedule of Activities

